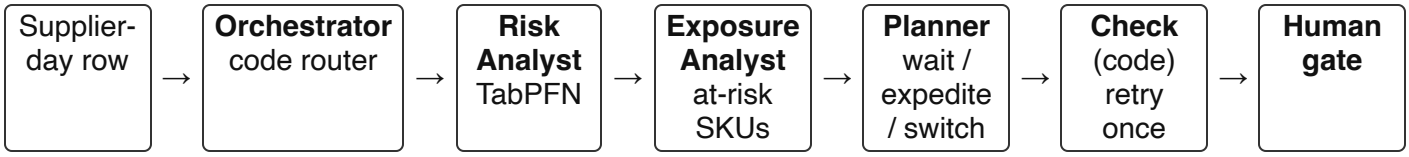
Supplier disruption detection and replenishment agents

Lakshya Sharma · Track A (Agentic / Multi-Agent AI) · Scenario S1 (Supply Chain Risk)
github.com/lakshya082001/FMs-AgenticAI

1. Problem & Domain

Problem. Detect supplier disruptions early from operational data, then recommend a replenishment action (wait, expedite or switch supplier) before stock runs out, with a human sign-off on risky plans.

Why Track A. B (fine-tuning) has no language task to adapt, C (classical ML) would stop at a risk score when the value is the action, and D (RAG) has no document corpus. A single LLM or a bare classifier cannot give tool use, an independent check on the plan, and a human gate.



If p is below the threshold the run ends with a "not at risk" summary. The gate fires on borderline p, plan cost above 10,000, or a failed check.

2. Approach & Algorithm Decisions

Why a model, not agents alone. Spotting a disruption needs relations between many signals (late rates, in-transit delays, volume), which an LLM cannot compute reliably. Those relations can be encoded in a graph database or learned by a model. A graph needs more exhaustive relational data and build time, so as a quick placeholder we use **TabPFN**, a foundation model for tabular data: it gives instant probabilistic predictions on small data with no hyperparameter tuning.

Decision	Chosen	Rejected, and why
Detector	TabPFN on 11 trailing-window features	Graph DB (data and time). XGBoost (valid baseline, deliberately not run, so no claim against it). LSTM/Chronos (3.6K rows, windows already summarise history).
Data	Real DataCo orders plus simulated disruption episodes	Raw labels: no temporal signal (lane autocorrelation –0.02 to –0.06). Fully synthetic data with planted rules: circular.
Orchestration	LangGraph: explicit state, conditional edges, retry loop, native human interrupt	CrewAI, AutoGen: sequential or chat flows fit a retry-and-gate loop less cleanly (judgement, not benchmarked).
Decomposition	3 LLM agents (detect, exposure, action) with a code router and a code check	One LLM with tools: no independent verification. LLM critic: a numeric check is cheaper and cannot hallucinate.
Planner	LLM picks from a tool-computed cost table; it never computes numbers	Pure argmin or OR-Tools: optimal but no rationale. The check keeps the LLM within 10% of the cheapest option.

Agent	One-liner
Orchestrator (code)	Routes after every step and logs each decision with its reason.
Risk Analyst	Validates the row, runs the pickled TabPFN, writes a hedged evidence summary.
Exposure Analyst	Lists the supplier's SKUs whose days of cover fall short of the disruption-adjusted lead time.
Planner	Chooses wait, expedite or switch per SKU; the code check verifies numbers and cost-optimality.

Silent failures found and handled. The LLM wrote "0.27 is below 0.25", so alert status and hedge wording are now computed in code. It missed the cheapest option on 3 of 15 early decisions (one about 15K worse),

so an optimality check with one retry was added. Injected wrong numbers are caught by the number check, and unresolved failures go to a human.

Related work. arXiv 2601.09680 runs 7 CrewAI/GPT-4o agents triggered by news, over a Neo4j multi-tier supplier graph with a rule-weighted risk score, scored on 30 expert scenarios (F1 0.90 to 0.99). We differ by triggering on structured operational data with a calibrated probability, a deterministic verification step and an executable human gate; they are stronger on multi-tier graph reasoning and expert-labelled evaluation. The Neo4j article layers agents over a supply-chain graph database so they query relationships directly (I could not open the full text and rely on its description). We replace the graph with a model for speed, and would add the graph back for multi-tier reasoning.

3. Results & Error Analysis

Data. DataCo orders (180K rows) with 8 simulated suppliers, 118 SKUs and seeded disruption episodes. Target: supplier is inside an episode that day. Features: late rate over 3/7/14/30/90 days, 7-day volume, two deviations, days since last alert, and the count and share of in-transit orders past their promised date. Temporal split: train 3,656 rows (Apr 2015 to Jun 2016), validation 1,472, test 2,184 (Jan to Sep 2017).

	AUC	Avg precision	Brier	ECE	Base rate	Best single feature AUC
Validation	0.659	0.298	0.121	0.071	0.145	0.689
Test	0.652	0.402	0.172	0.091	0.234	0.664

At threshold 0.25 (best F1 on validation) 27 of 35 test episodes are detected, with median delay 5 days and a 17.4% false-alarm rate. **TabPFN does not beat the single feature od_share**; the gain comes from the in-transit overdue signal, not the model. With one seed and 35 episodes, differences near 0.01 AUC are noise. The agent runs end to end in 40 to 46 s (about 30 s is single-row CPU inference) with 1 to 4 LLM calls and 2K to 4K tokens; eight committed traces cover success, borderline, retry, false alarm, invalid input and fault injection.

Failure point	What happens
Long-range temporal dependence	Windows stop at 90 days and episodes are memoryless, so seasonality and recurring supplier fragility are not modelled or tested.
Train-to-test shift	Disruption rate rises from 15% to 23%, which hurts calibration.
Detection lag	Late deliveries only show on completion, so alerts trail episode start by about 5 days.
False alarms	A supplier at p = 0.71 that was not disrupted still received a plan.
Planning assumptions	Alternate suppliers are assumed undisrupted and exposure is single-tier.

4. Production & Limitations

Step	Plan
Model	Export the fitted TabPFN weights and host them (or use the hosted API). Score suppliers in a daily GPU batch and cache results, since single-row CPU inference takes about 30 s.
Graph (optional)	Store the supplier network in Neo4j for multi-tier exposure once data allows.
Agents	Deploy the LangGraph app as a service with a persistent checkpointer so human approvals survive restarts. Add per-node latency and cost tracing (today only per-run totals are recorded).
Monitoring	Track alert rate, feature drift and calibration once labels mature.

Limitations. Disruptions are simulated, so results measure detection of simulated events, not real-world accuracy. The TabPFN v3.5 weights carry a non-commercial license. Next step: a what-if agent that sweeps features to show what would clear an alert.